KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Verslag: Race conditions

Geavanceerde computerarchitectuur

Robbe Hovaert
Lukas Hoste

Master in de ingenieurswetenschappen
Academiejaar 2024-2025

Context

Het doel van dit labo is om kennis te maken met race conditions op de GPU. Om dit te doen vinden we het maximum in een array. In een eerste versie deden we dit door gebruik te maken van atomics om daarna over te schakelen naar een reductie gebaseerd algoritme met synchronisatie binnen blocks.

Methodologie

Atomicmax

In onze eerste implementatie gebruiken wij atomicMax om het maximum te vinden. Elke thread krijgt een index in de array toegewezen en kijkt met atomicMax of de waarde op deze index groter is dan de huidige maximale waarde.

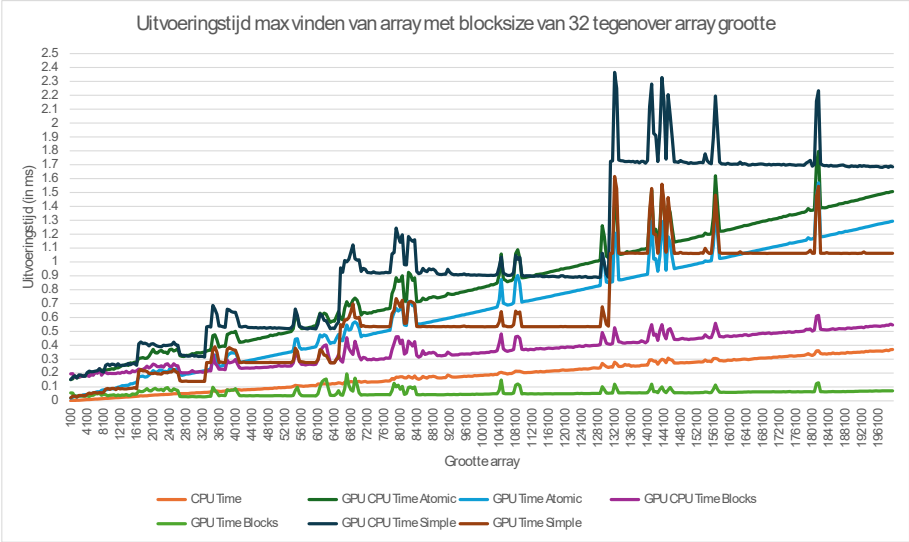
Single block reduction

Als een tweede stap hebben wij een reductie algoritme geïmplementeerd dat werkt op maar één block. Deze restrictie hebben wij gemaakt omdat dit synchronisatie eenvoudiger maakt met de `__syncthreads()` functie van CUDA. Elke thread staat initieel op een even index in de array. De eerste iteratie vergelijkt elke thread zijn waarde met de waarde één naar rechts en zet het maximum op zijn eigen index. De volgende iteratie schakelt de helft van de overblijvende threads zich uit (één thread aan, één thread uit, enz..) en wordt de hoeveelheid indices verder kijken maal twee gedaan. Het resultaat hiervan is dat uiteindelijk index nul in de array het maximum wordt. We stoppen vanaf er nog maar één thread over is en deze gedaan heeft. We aligneren de array ook tot de eerste macht van twee die groter of gelijk is aan de grootte van de array.

Multiple block reduction

Als de laatste stap hebben wij de single block implementatie uitgebreid naar meerdere blocks in combinatie met een algoritme dat recursief eenvoudiger werkt.

Als eerste wordt de array vergroot tot het dichtsbijzijnde veelvoud van de block size * 2. In de kernel wordt gebruik gemaakt van twee shared memory buffers. Elk block krijgt een stuk van de array waarin het maximum moet gevonden worden. Elk block begint door zijn stuk van de array naar de eerste shared memory buffer te schrijven. Vervolgens wordt de eerste buffer als `src` benoemd, de tweede als `dst`. Elke thread staat op een even index van src en kijkt naar het volgende element. Het max van deze twee schrijft de thread op de `threadId`-de plaats in de `dst` buffer. De threads synchroniseren en de grootte van de buffers wordt logisch als gedeeld door twee gezien. De rollen van `src` en `dst` worden nu omgedraaid. Het resultaat hiervan is dat er in `src` een array zit met hetzelfde maximum, maar met de helft van de lengte. Dit herhalen we tot er maar één element is per block. De eerste thread in elke block schrijft dan het maximum van zijn block weg op de `blockIdx`-de plaats in de originele array. De kernel kan dus zelf ook gezien worden als een iteratie van dit algoritme dat de array reduceert met een factor van `blockSize`. Het algoritme aan de CPU kant is hetzelfde als op de GPU.



Figuur 1: Block size tegenover executietijd in ms (gemiddelde van 100 uitvoeringen)

Meer elementen dan threads

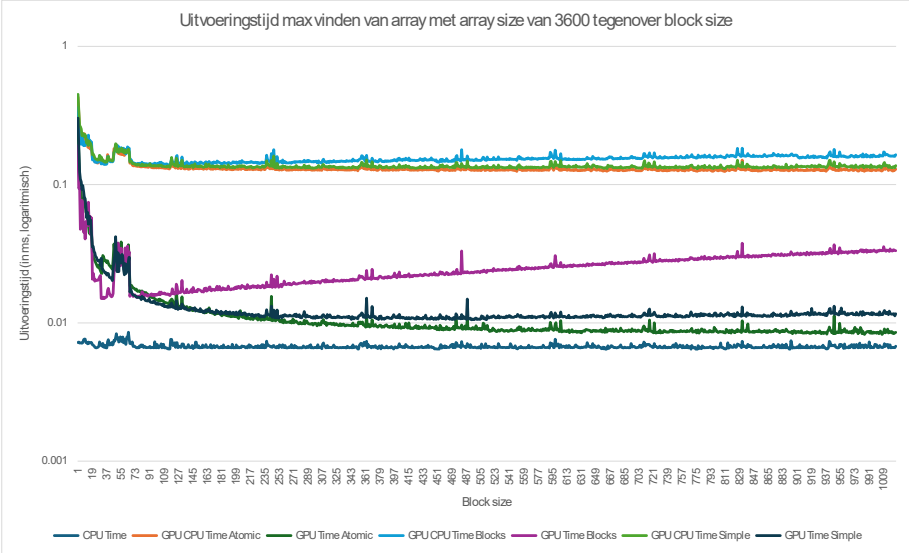
De single block variant werd aangepast zodat deze werkt met een array grootte groter dan 2048 (MaxBlockSize$*2$). Deze variant begint door de array aan te passen naar een grootte die voldoet aan 2^n. In deze versie wordt gebruik gemaakt van een roundrobin methode die start achter de block en zo de array reduceert tot een grootte waarmee gewerkt kan worden.

Observaties en resultaten

Wij hebben aan de bovenstaande implementatie timing-informatie toegevoegd. Wij houden de tijd dat de GPU nodig heeft om dit uit te voeren (tijd zonder memcpy) bij (GPU Time), de tijd op de CPU om de data te verplaatsen en de operatie op de gpu uit te voeren (GPU CPU Time) en een referentietijd van een puur CPU implementatie met een for-loop (CPU Time).

De eerste test die we hebben uitgevoerd is het variëren van de grootte van de array. In Figuur 1 is te zien dat de benodigde tijd op de CPU lineair stijgt, ook de atomic variant stijgt lineair, maar sterker. Dit vermoedelijk omdat bij een grotere array steeds vaker moet gewacht worden tot toegang tot de data kan verkregen worden. In de simpele reductie variant is te zien dat de tijd regelmatig een stap omhoog gaat en dan een tijd ongeveer hetzelfde blijft. Dit heeft te maken met het feit dat de array gepad wordt om aan 2^n te voldoen. Hoe groter de array wordt hoe groter het effect hier ook van wordt. In de versie die werkt met meerdere blocks werd ervoor gezorgd dat deze padding niet even sterk nodig is. Hierdoor blijft de benodigde tijd op de GPU vrij constant, de tijd met de CPU ingerekend stijgt echter wel lineair omdat meer data gekopieerd moet worden naar de GPU en omdat er meer blocks zijn en dus ook meer synchronisatie.

Als tweede werd het effect van de block size op de verschillende methoden getest. Zoals te zien in Figuur 2 daalt de block size tot een limiet van performantie bereikt wordt rond 400 bij de atomic variant. Deze limiet is waarschijnlijk doordat veel gewacht moet worden voor toegang tot data. De simpele variant bereikt ook een limiet rond 400, maar stijgt



Figuur 2: Block size tegenover executietijd in ms (gemiddelde van 100 uitvoeringen)

hierna lichtjes. Dit is waarschijnlijk doordat meer synchronisatie nodig is. De variant die met meerdere blocks werkt bereikt een limiet rond 32 à 64 (het aantal cores per SM op de gebruikte GPU). Hierna stijgt de uitvoeringstijd, dit omdat er in deze methode meer synchronisatie gebeurt en een grotere block size voor meer synchronisatie zorgt.

Conlcusie

In dit labo hebben wij kennis gemaakt met race conditions op de GPU en de problemen die hierbij komen. We besluiten dat het maximum zoeken met een gedistribueerd algoritme zeker sneller kan op de GPU dan op de CPU, maar dat het extra werk dat op de CPU gebeurt om alles klaar te zetten zeker in rekening gehouden moet worden. We hebben ook het belang opgemerkt van het gebruiken van een eenvoudig algoritme dat eenvoudig beweegt van de GPU naar de CPU en omgekeerd. Onze experimenten met het aanpassen van de block size en array size gaven ons de verwachtte resultaten.

A CODE

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <chrono>

#define N 200000
#define MAX_BLOCK_SIZE 1024


int cpuMax(int* arr, int n, float * time) {
    const auto start = std::chrono::steady_clock::now();
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    const auto end = std::chrono::steady_clock::now();
    *time = std::chrono::duration<float, std::milli>(end -
    ↪  start).count();
    return max;
}


__global__ void __gpuMaxAtomicVariant(int* arr, int n, int
↪  amountThreads,int* maxVal) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    while (i < n) {
        atomicMax(maxVal, arr[i]);
        i += amountThreads;
    }
}



int gpuMaxAtomixVariant(int* arr, int n, int blockSize, float *
↪  gpuCpuTime, float * gpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();

        // Allocate memory on device
    int *gpuArr;
    int *gpuMaxVal;
    cudaMalloc(&gpuArr, n * sizeof(int));
```

```
    cudaMalloc(&gpuMaxVal, sizeof(int));



    // Copy input array to device
    cudaMemcpy(gpuArr, arr, n * sizeof(int), cudaMemcpyHostToDevice);

    cudaEventRecord(start);

    // Configure kernel launch parameters
    // int blocksPerGrid = (N + THREADS_PER_BLOCK - 1) /
    ↪  THREADS_PER_BLOCK; // Ensure all elements are covered

    // Launch the kernel with blocks and threads
    __gpuMaxAtomicVariant<<<1, blockSize>>>(gpuArr, n, blockSize,
    ↪  gpuMaxVal);
    cudaDeviceSynchronize(); // Ensure kernel execution completes

    // Copy result back to host
    int maxVal;
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(&maxVal, gpuMaxVal, sizeof(int), cudaMemcpyDeviceToHost);
    cudaFree(gpuArr);
    cudaFree(gpuMaxVal);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

    return maxVal;
}

__global__ void __gpuSimpleMaxReduction(int* arr, int n) {
    int id = blockIdx.x * blockDim.x + threadIdx.x;
    int index = id * 2;
    if (n >= (blockDim.x * 2)) {
        // oh no! we have too many elements. this can hovever easly be
        ↪  done
        // each element from the starting of MAX_BLOCK_SIZE is handled in
        ↪  a round robin style by all the threads in a for loop, thus
        ↪  reducing the amount of elements to MAX_BLOCK_SIZE
        int initialTroublingIndex = blockDim.x * 2;
        int* ogElem = arr + index;
```

```
        int correctingIndex = initialTroublingIndex + id;
        while (correctingIndex < n) {
            if (*ogElem < arr[correctingIndex]) {
                *ogElem = arr[correctingIndex];
            }
            correctingIndex += blockDim.x; // we don't multiply by
            ↪  because we don't devide and concor

        }
        n = blockDim.x * 2; // we have reduced the amount of elements to
        ↪  MAX_BLOCK_SIZE
        __syncthreads();
    }

    // we start with one thread per two elements
    // each iteration halve the amount of threds is useless.

    int skip = 1;

    while(n > 1) {
         // skip is also 2^i
         if (index % skip == 0) {
            if (arr[index] < arr[index + skip]) {
                arr[index] = arr[index + skip];
            }
            skip *= 2;
        }
        __syncthreads();
        n = (n + 1) / 2; // foor ceiling division
    }
}

int gpuSimpleMaxReduction(int* arr, int n, int blockSize, float *
↪  gpuCpuTime, float * gpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();
    int allignedN = pow(2, ceil(log2(n)));
    int* allignedArr = (int*) malloc(allignedN * sizeof(int));
    for (int i = 0; i < n; i++) {
        allignedArr[i] = arr[i];
    }
    for (int i = n; i < allignedN; i++) {
        allignedArr[i] = 0;
    }

    // Allocate memory on device
    int *gpuArr;
```

```
    cudaMalloc(&gpuArr, allignedN * sizeof(int));

    // Copy input array to device
    cudaMemcpy(gpuArr, allignedArr, allignedN * sizeof(int),
    ↪  cudaMemcpyHostToDevice);

    // Configure kernel launch parameters
    // int blocksPerGrid = (N + THREADS_PER_BLOCK - 1) /
    ↪  THREADS_PER_BLOCK; // Ensure all elements are covered

    // Launch the kernel with multiple blocks and threads
    //int amountIterations = log2(min(allignedN, blockSize * 2));
    cudaEventRecord(start);
    __gpuSimpleMaxReduction<<<1, blockSize>>>(gpuArr, allignedN);
    cudaDeviceSynchronize(); // Ensure kernel execution completes

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    // Copy result back to host
    int maxVal;
    cudaMemcpy(&maxVal, gpuArr, sizeof(int), cudaMemcpyDeviceToHost);
    cudaFree(gpuArr);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;

    free(allignedArr);
    return maxVal;
}

void printArr(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

__global__ void __gpuAdvancedMaxReduction(int* arr, int* output, int n) {
    __shared__ int buf1[2048]; // max 1024 threads per block, so 2048 is
    ↪  enough
    __shared__ int buf2[2048];
```

```
    n = min(n, blockDim.x * 2); // we keep in our local block (the length
    ↪  of the small array)

    int* shiftedArray = arr + (blockIdx.x * blockDim.x * 2); // base of
    ↪  this block
    int index = threadIdx.x * 2;

    if(threadIdx.x == 0) {
        for (int i = 0; i < n; i++) {
            buf1[i] = shiftedArray[i];
        }
    }

    // sync threads
    __syncthreads();


    int* src = buf1;
    int* dst = buf2;

    // we always reduce to half the size
    while(n != 1) {
        if (index < n) { // out of bounds in this iteration, disable
            if (index + 1 < n) {
                dst[index / 2] = max(src[index], src[index + 1]);
            } else {
                // possible last element when the array is not even. it
                ↪  is always "bigger"
                dst[index / 2] = src[index];
            }
        }


        int* temp = src;
        src = dst;
        dst = temp;
        // in in half with round up! (the round down by default
        ↪  counteracts the +1)
        n = (n + 1) / 2;
        __syncthreads();
    }

    if (threadIdx.x == 0) {
        output[blockIdx.x] = src[0];
    }
}
```

```
int gpuAdvancedMaxReduction(int* arr, int n, int threadsPerBlock, float *
↪  gpuCpuTime, float * gpuTime) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    const auto startCPU = std::chrono::steady_clock::now();
    // we need to round up to the nearse MAX_BLOCK_SIZE
    int allignedN = (int)ceil((double)n / (double)(threadsPerBlock * 2))
    ↪  * threadsPerBlock * 2;
    //int allignedN = pow(2, ceil(log2(n)));
    int* allignedArr = (int*) malloc(allignedN * sizeof(int));

    for (int i = 0; i < n; i++) allignedArr[i] = arr[i];
    for (int i = n; i < allignedN; i++) allignedArr[i] = INT_MIN; // Use
    ↪  INT_MIN for safety

    int *gpuArr1, *gpuArr2;
    cudaMalloc(&gpuArr1, allignedN * sizeof(int));
    cudaMalloc(&gpuArr2, (allignedN ) * sizeof(int)); // Allocate only
    ↪  needed space

    cudaMemcpy(gpuArr1, allignedArr, allignedN * sizeof(int),
    ↪  cudaMemcpyHostToDevice);

    cudaEventRecord(start);

    int currentN = allignedN;
    //printArr(allignedArr, allignedN);

    while(currentN > 1) {
        int amountBlocks = (int)ceil((double)currentN /
        ↪  (double)(threadsPerBlock * 2));
        //printf("Amount blocks: %d\n", amountBlocks);
        //printf("Amount threads: %d\n", threadsPerBlock);
        //printf("Current N: %d\n", currentN);
        __gpuAdvancedMaxReduction<<<amountBlocks,
        ↪  threadsPerBlock>>>(gpuArr1, gpuArr2, currentN);
        cudaDeviceSynchronize();


        // each block became one value
        currentN = amountBlocks;


        int* temp = gpuArr1;
        gpuArr1 = gpuArr2;
        gpuArr2 = temp;
        //cudaError_t err = cudaMemcpy(allignedArr, gpuArr1, currentN *
        ↪  sizeof(int), cudaMemcpyDeviceToHost);
```

```
        //if (err != cudaSuccess) {
        //    printf("CUDA memcpy failed: %s\n",
        ↪  cudaGetErrorString(err));
        //}
        //printArr(allignedArr, currentN);
    }

    int maxVal;
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaMemcpy(&maxVal, gpuArr1, sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(gpuArr1);
    cudaFree(gpuArr2);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    *gpuCpuTime = std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
    *gpuTime = milliseconds;
    free(allignedArr);


    return maxVal;
}

// array shuffle function
void shuffle(int* arr, int n) {
    // own version of shuffle
    for (int i = 0; i < n; i++) {
        int j = rand() % n;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

int main() {
    int testArray[] = {
        1, 3, 10, 2, 7, 2 ,0, 21, 20 ,2, 5, 0
    };

    // generate array starting at 0 counting to size-1
    int testArray2[N];
    for (int i = 0; i < N; i++) {
        testArray2[i] = i;
    }
```

```
// shuffle the array
shuffle(testArray2, N);

// print element 1 of array
//printf("Element 1: %d\n", testArray2[1]);


int n = sizeof(testArray) / sizeof(int);
float cpuTime = 0;

float gpuTimeAtomic = 0;
float gpuCpuTimeAtomic = 0;

float gpuTimeBlocks = 0;
float gpuCpuTimeBlocks = 0;

// for loop that tests all of the methods with different array sizes
↪  and prints their times all together
printf("Array size, Block size, CPU Time, GPU CPU Time Atomic, GPU
↪  Time Atomic, GPU CPU Time Blocks, GPU Time Blocks, GPU CPU Time
↪  Simple, GPU Time Simple\n");
# define FIX_ARRSIZE  3600
//# define FIX_BLOCKSIZE 32

# ifdef FIX_ARRSIZE
for (int i = FIX_ARRSIZE; i <= FIX_ARRSIZE; i+= 500) {
#else
for(int i = 100; i < N; i+= 500) {
#endif

#ifdef FIX_BLOCKSIZE
    for (int k = FIX_BLOCKSIZE; k <= FIX_BLOCKSIZE; k += 1) {
#else
    for (int k = 1; k <= 1024; k += 1) {
#endif
        float cpuTimeTotal = 0;
        float gpuCpuTimeAtomicTotal = 0;
        float gpuTimeAtomicTotal = 0;
        float gpuCpuTimeBlocksTotal = 0;
        float gpuTimeBlocksTotal = 0;
        float gpuTimeSimpleTotal = 0;
        float gpuCpuTimeSimpleTotal = 0;
        for(int j = 0; j < 100; j++) {
            float cpuTime = 0, gpuCpuTimeAtomic = 0, gpuTimeAtomic =
            ↪  0, gpuCpuTimeBlocks = 0, gpuTimeBlocks = 0,
            ↪  gpuTimeSimple = 0, gpuCpuTimeSimple = 0;

            int cpuResult = cpuMax(testArray2, i, &cpuTime);
```

```
            int gpuAtomicResult = gpuMaxAtomixVariant(testArray2, i,
            ↪  k, &gpuCpuTimeAtomic, &gpuTimeAtomic);
            int gpuAdvancedMaxresult =
            ↪  gpuAdvancedMaxReduction(testArray2, i, k,
            ↪  &gpuCpuTimeBlocks, &gpuTimeBlocks);
            int gpuSimpleMaxResult =
            ↪  gpuSimpleMaxReduction(testArray2, i, k,
            ↪  &gpuCpuTimeSimple, &gpuTimeSimple);

            if (cpuResult != gpuAtomicResult || cpuResult !=
            ↪  gpuAdvancedMaxresult || cpuResult !=
            ↪  gpuSimpleMaxResult) {
                printf("Error at %d, %d\n", i, k);
                printf("CPU: %d, GPU Atomic: %d, GPU Advanced: %d,
                ↪  GPU Simple: %d\n", cpuResult, gpuAtomicResult,
                ↪  gpuAdvancedMaxresult, gpuSimpleMaxResult);
                return -1;
            }
            cpuTimeTotal += cpuTime;
            gpuCpuTimeAtomicTotal += gpuCpuTimeAtomic;
            gpuTimeAtomicTotal += gpuTimeAtomic;
            gpuCpuTimeBlocksTotal += gpuCpuTimeBlocks;
            gpuTimeBlocksTotal += gpuTimeBlocks;
            gpuTimeSimpleTotal += gpuTimeSimple;
            gpuCpuTimeSimpleTotal += gpuCpuTimeSimple;
        }

        printf("%d, %d, %f, %f, %f, %f, %f, %f, %f\n",
            i,
            k,
            cpuTimeTotal / 100,
            gpuCpuTimeAtomicTotal / 100,
            gpuTimeAtomicTotal / 100,
            gpuCpuTimeBlocksTotal / 100,
            gpuTimeBlocksTotal / 100,
            gpuCpuTimeSimpleTotal / 100,
            gpuTimeSimpleTotal / 100
        );
    }


}


// for (int i = 1; i < min(N, 1024); i++) {
//     if (gpuAdvancedMaxReduction(testArray2, N, i) !=
↪  cpuMax(testArray2, N)) {
//         printf("Error at %d\n", i);
//     }
```

```
    // }
    return 0;
}
```